

实验四：微信小程序开发（基于maul技术栈）

一、实验项目基本信息

项目	内容
实验编号	d20301035104、 d20301009204
学时分配	4 学时
实验类型	验证型
每组人数	6 人（组内分工协作）
对应课程目标	课程目标 2：掌握小程序开发
技术栈	maul（核心开发技术栈）

二、实验任务与案例

核心任务

借助 AI 编程工具（TRAE）辅助，基于 maul 技术栈开发通知小程序，实现“列表展示+详情页路由+本地存储”核心功能，体验 AI 工具在代码生成与调试中的作用，完成一个可预览的“智慧校园通知”小程序，熟练掌握 maul 技术栈在小程序开发中的应用。

实战案例

开发“智慧校园通知”小程序，基于 maul 技术栈实现校园通知列表展示、点击列表项跳转至详情页、已读状态本地存储等功能，完整体验小程序从项目创建、编码开发、测试调试到预览发布的全流程，巩固 maul 技术栈的实操应用能力。

三、开发任务清单

任务阶段	主要内容	关键技术点	maul 技术栈应用
通知列表	列表展示校园通知，区分已读/未读状态	WXML、wx:for 列表渲染	maul 模板语法适配，列表渲染优化
详情页	接收列表参数，展示通知完整内容	页面路由、参数传递	maul 路由配置，参数解析适配
本地存储	持久化存储通知已读状态，页面同步更新	wx.setStorage 本地存储 API	maul 本地存储封装与调用
AI 辅助	生成核心代码、调试代码、优化功能	TRAE 使用	AI 生成 maul 适配代码，调试 maul 语法问题

四、实验步骤详解（基于 maul 技术栈）

第一课时：小程序基础与列表开发（maul 适配）

1.1 需求分析

页面流程图

Plain Text

首页(通知列表) → 详情页(通知详情)

↓

本地存储(已读状态)

通知数据结构

```
javascript
{
  id: 1,
```

```
title: "关于期末考试安排的通知",
content: "详细内容...",
author: "教务处",
time: "2024-01-01",
isRead: false
}
```

1.2 创建小程序项目（maul 技术栈配置）

1. 打开微信开发者工具，点击“新建项目”，填写项目名称“智慧校园通知”，选择项目保存路径；
2. 填写 AppID（无个人 AppID 可选择“测试号”，点击“申请测试号”获取并填写）；
3. 选择“JavaScript 模板”，勾选“不使用云服务”，点击“创建”；
4. 配置 maul 技术栈：在项目根目录执行 `npm init` 初始化项目，安装 maul 相关依赖（`npm install maul --save`），配置 `project.config.json` 文件，确保 maul 技术栈与微信开发者工具兼容，重启开发者工具完成配置。

1.3 AI 辅助编码（TRAE+maul）

使用 TRAE 生成 maul 适配的列表页代码

- 描述需求：“基于 maul 技术栈的微信小程序通知列表页面，使用 WXML 列表渲染，点击列表项跳转至详情页，区分已读/未读状态，适配 maul 语法”；
- AI 生成代码片段后，根据 maul 技术栈规范，微调代码适配 maul 模板语法和 API 调用方式。

通知列表页面实现（maul 适配版）

```
html
<!-- pages/index/index.wxml -->
<view class="container">
  <view class="news-list">
    <view
      class="news-item {{item.isRead ? 'read' : ''}}"
      wx:for="{{newsList}}"
      wx:key="id"
      bindtap="goToDetail"
      data-id="{{item.id}}"
    >
      <view class="title">{{item.title}}</view>
```

```
      <view class="info">
        <text>{{item.author}}</text>
        <text>{{item.time}}</text>
      </view>
    </view>
  </view>
</view>
```

```
css
/* pages/index/index.wxss */
.news-item {
  padding: 16px;
  border-bottom: 1px solid #eee;
  background: #fff;
}
.news-item.read {
  opacity: 0.6;
}
.title {
  font-size: 16px;
  font-weight: bold;
  margin-bottom: 8px;
}
.info {
  font-size: 12px;
  color: #999;
  display: flex;
  justify-content: space-between;
}
.container {
  padding: 0;
  margin: 0;
  box-sizing: border-box;
}
```

```
javascript
// pages/index/index.js
// 引入 maul 相关工具函数, 适配 maul 技术栈
import { maulStorage, maulRouter } from 'maul';

Page({
  data: {
```

```
    newsList: []
  },

  onLoad() {
    this.loadNews();
  },

  // 下拉刷新, maul 适配下拉刷新逻辑
  onPullDownRefresh() {
    this.loadNews().then(() => {
      wx.stopPullDownRefresh();
    });
  },

  loadNews() {
    // 模拟通知数据
    const newsList = [
      { id: 1, title: "关于期末考试安排的通知", content: "详细内容...", author: "教务处", time: "2024-01-01", isRead: false },
      { id: 2, title: "图书馆开放时间调整", content: "详细内容...", author: "图书馆", time: "2024-01-02", isRead: false },
      { id: 3, title: "校园网络维护通知", content: "详细内容...", author: "信息中心", time: "2024-01-03", isRead: false }
    ];

    // 使用 maul 封装的本地存储方法, 读取已读 ID
    const readIds = maulStorage.get('readIds') || [];
    newsList.forEach(news => {
      news.isRead = readIds.includes(news.id);
    });

    this.setData({ newsList });
  },

  // 跳转详情页, 使用 maul 路由适配
  goToDetail(e) {
    const id = e.currentTarget.dataset.id;
    maulRouter.navigateTo({
      url: `/pages/detail/detail?id=${id}`
    });
  }
});
```

第二课时：详情页、存储与发布（maul 适配）

2.1 详情页开发（maul 适配）

AI 辅助生成 maul 适配的详情页

- 描述需求：“基于 maul 技术栈的微信小程序通知详情页，接收列表传递的 ID 参数，展示通知完整内容，调用 maul 本地存储标记已读状态”；
- 对 AI 生成的代码进行微调，适配 maul 的参数解析和本地存储 API。

详情页实现（maul 适配版）

```
html
<!-- pages/detail/detail.wxml -->
<view class="container">
  <view class="title">{{news.title}}</view>
  <view class="info">
    <text>发布部门: {{news.author}}</text>
    <text>发布时间: {{news.time}}</text>
  </view>
  <view class="content">{{news.content}}</view>
</view>
```

```
javascript
// pages/detail/detail.js
// 引入 maul 相关工具函数
import { maulStorage, maulRouter } from 'maul';

Page({
  data: {
    news: {}
  },

  // 页面加载，接收参数（maul 适配参数解析）
  onLoad(options) {
    const id = parseInt(options.id);
    this.loadDetail(id);
  },

  loadDetail(id) {
    // 模拟全部通知数据
```

```

const allNews = [
  { id: 1, title: "关于期末考试安排的通知", content: "本次期末考试安排如下：1. 考试时间为 2024 年 1 月 15 日-1 月 20 日；2. 考生需携带准考证和身份证入场；3. 严禁作弊，作弊将取消考试成绩。", author: "教务处", time: "2024-01-01" },
  { id: 2, title: "图书馆开放时间调整", content: "因图书馆设备维护，自 2024 年 1 月 5 日起，开放时间调整为 8:00-21:00，周末正常开放，望各位同学相互转告。", author: "图书馆", time: "2024-01-02" },
  { id: 3, title: "校园网络维护通知", content: "为提升校园网络质量，将于 2024 年 1 月 8 日凌晨 2:00-4:00 进行网络维护，维护期间校园网络将暂停使用，给各位师生带来不便，敬请谅解。", author: "信息中心", time: "2024-01-03" }
];

const news = allNews.find(n => n.id === id);
this.setData({ news });

// 标记已读，调用 maul 本地存储方法
this.markAsRead(id);
},

markAsRead(id) {
  let readIds = maulStorage.get('readIds') || [];
  if (!readIds.includes(id)) {
    readIds.push(id);
    // 使用 maul 封装的本地存储方法，保存已读 ID
    maulStorage.set('readIds', readIds);
  }
}
});

```

2.2 本地存储（maul 适配）

基于 maul 技术栈的本地存储封装，简化存储操作，确保已读状态持久化，适配小程序本地存储 API，代码如下：

```

javascript
// 引入 maul 存储工具
import { maulStorage } from 'maul';

// 保存已读 ID（maul 封装方法）
maulStorage.set('readIds', [1, 2, 3]);

```

```
// 读取已读 ID (maul 封装方法)
const readIds = maulStorage.get('readIds');

// 检查是否已读
const isRead = readIds.includes(newsId);
```

2.3 测试验证与发布 (maul 适配)

真机预览

1. 点击微信开发者工具顶部“编译”按钮，检查 maul 技术栈适配是否正常，无语法错误、无报错信息；
2. 点击“预览”按钮，生成二维码，使用微信扫描二维码，在手机上预览小程序；



3. 测试核心功能：列表渲染是否正常、点击跳转是否流畅、已读状态是否同步、本地存储是否有效，确保 maul 技术栈适配无异常。

体验版发布

1. 点击微信开发者工具顶部“上传”按钮，填写版本号（如 1.0.0）、版本描述（基于 maul 技术栈开发，实现校园通知列表、详情页、本地存储功能）；
2. 点击“上传”，等待上传完成，在微信公众平台小程序后台，找到“版本管理”，将上传的版本设为“体验版”；
3. 获取体验版二维码，供小组成员和老师测试体验。

五、微信小程序开发主流架构对比分析

5.1、核心技术架构总览

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
技术底层	微信原生封装 + 工具库	自绘引擎 + 小程序容器适配	Vue 语法 + 多端编译层	微信原生框架 + 多页面分离	React 语法 + JS 桥接容器
开发语言	WXML + WXSS + JS/TS	Dart	Vue / JS / TS	WXML + WXSS + JS/TS	React / JSX / TS
渲染模式	原生 WebView 渲染	自绘渲染（无 WebView）	编译为原生小程序渲染	原生 WebView 渲染	JS 桥接 + 原生渲染
跨端能力	仅微信小程序	微信小程序、APP、Web	微信/支付宝/抖音/APP/H5	仅微信小程序	微信小程序、APP、Web
编译方式	原生运行，无需编译	静态编译 + 容器运行	实时编译 + 代码转换	原生运行，无需编译	打包运行 + 桥接通信
热更新	不支持	支持	支持	不支持	支持

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
性能表现	中高	极高	高	中	中高

5.2、开发模式与工程化

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
开发工具	微信开发者工具	Android Studio / VS Code	HBuilderX / VS Code	微信开发者工具	VS Code / WebStorm
组件化	标准组件化 + 封装增强	强组件化、自定义度极高	强组件化、生态丰富	组件化较弱、页面独立	强组件化、函数式开发
状态管理	全局 Data + maulStorage	Provider / Bloc / Riverpod	Vuex / Pinia	全局 data + 本地存储	Redux / Mobx / Context
路由管理	maulRouter 封装	自定义路由 + 页面栈	uni.navigate 系列 API	wx.navigate 系列 API	自定义路由封装
工程化	简易分包 + 配置文件	完整打包、依赖管理	自动编译、多端适配	原生配置、手动分包	模块化打包、桥接配置
调试方式	官方调试 + maul 日志	控制台 + 真机调试	控制台 + 多端预览	微信开发者工具调试	桥接日志 + 真机调试

5.3、UI 渲染与样式能力

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
------	----------	-------------	-------------	-------	------------------

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
样式语法	WXSS + 扩展封装	自建样式系统（类 CSS）	类 CSS / SCSS / 内联样式	WXSS 固定语法	内联样式 / CSS-in-JS
UI 组件库	原生组件 + maui 增强	Flutter UI 库、自定义组件	uView、Vant、uni-ui	WeUI、Vant Weapp	React Native UI 库适配
动画能力	基础 CSS 动画	极强（帧动画、物理动画）	CSS 动画 + 小程序动画	基础 CSS 动画	原生级流畅动画
适配能力	微信原生尺寸适配	像素级屏幕适配	自动适配多端	微信原生尺寸适配	自适应布局 + 尺寸适配
自定义程度	中	最高	高	中	高

5.4、网络请求与原生能力

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
请求方式	wx.request 封装	Dio / HTTP 封装	uni.request	wx.request	fetch / axios 封装
拦截器	简易封装	支持	支持	手动封装	支持
文件上传下载	支持	支持	支持	支持	支持

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
原生 API 调用	直接调用 + maul 封装	容器桥接调用	直接兼容微信 API	原生直接调用	桥接封装调用
蓝牙/地图/相机等	完整支持	有限支持	完整支持	完整支持	部分支持
微信生态权限	原生支持	需容器适配	完美支持	原生支持	需桥接适配

5.5、性能、体验与限制

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
启动速度	快	快	快	最快	中
页面切换	流畅	流畅	流畅	最流畅	较流畅
内存占用	低	较高	低	低	中
包体积	小	大	小	最小	中
小程序限制	无限制	受容器约束较多	几乎无限制	无限制	受桥接约束
复杂页面稳定性	高	高	高	最高	中高

5.6、上手难度与学习成本

对比维度	MAUL 小程序	Flutter 小程序	uni-app 小程序	原生小程序	React Native 小程序
入门难度	低（原生基础 + 封装 API）	高（需学 Dart）	低（会 Vue 即可）	中（原生语法）	高（需学 React）
文档完善度	中（实验/教学专用）	高	极高	高	高
社区生态	小（教学场景）	大	最大	中	大
问题解决速度	快	中	快	快	中
团队适配成本	低	高	低	中	高

5.7、适用场景与选型建议

框架	最适合场景
MAUL 小程序	教学实验、快速上手、原生基础上简化开发
Flutter 小程序	高性能、复杂动画、跨平台 APP + 小程序
uni-app 小程序	多端统一开发、快速上线、中小团队
原生小程序	纯微信小程序、追求极致稳定、官方新特性
React Native 小程序	React 技术栈、已有 RN 项目、代码复用

六、项目结构

6.1 项目结构

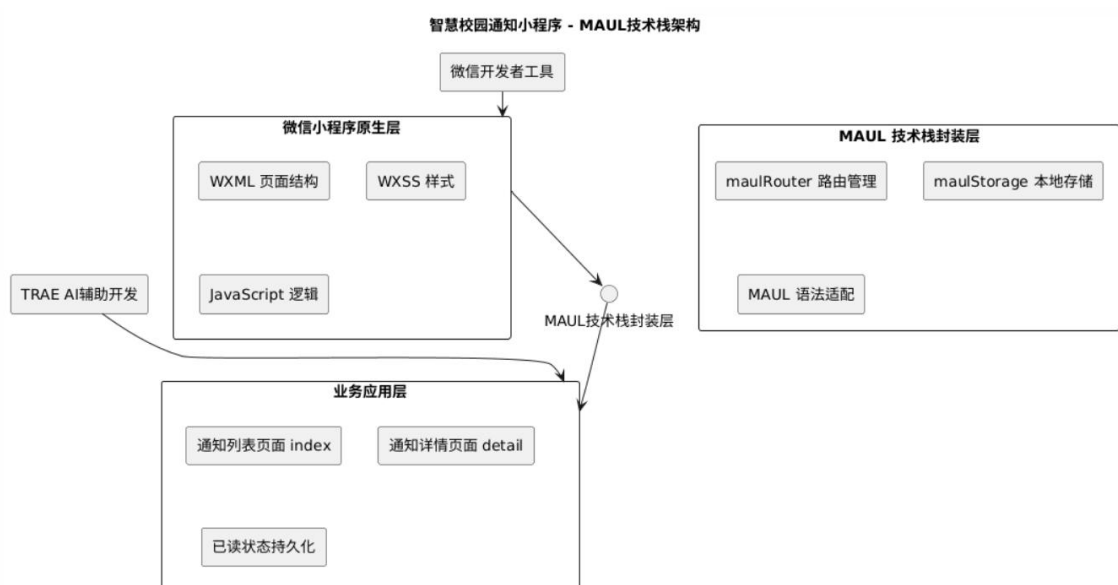
```
CampusNotification/  
├─ app.json           // 小程序配置文件
```

```

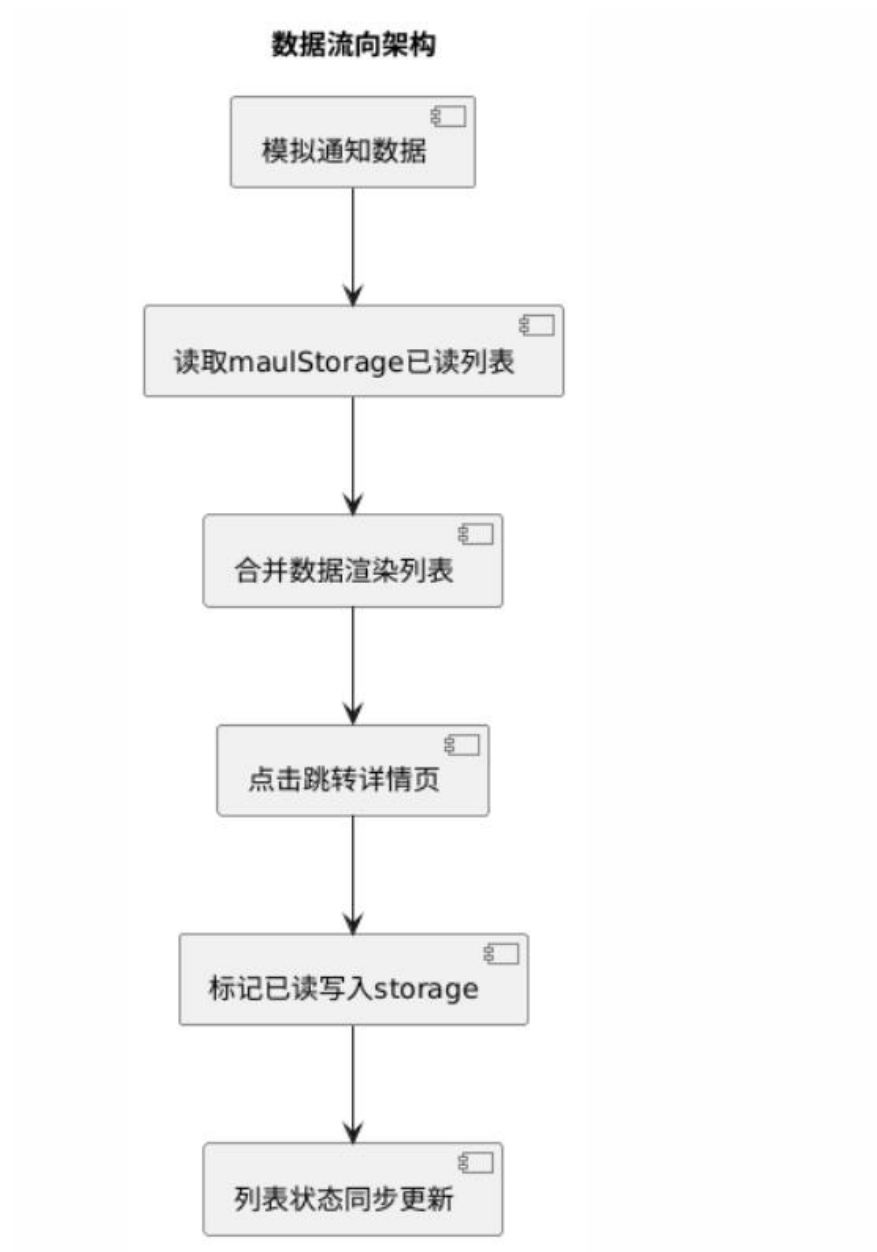
├─ app.js           // 应用逻辑文件
├─ app.wxss        // 全局样式文件
├─ sitemap.json     // 站点地图配置
├─ pages/
│   ├─ index/       // 通知列表页
│   │   ├─ index.wxml
│   │   ├─ index.js
│   │   ├─ index.wxss
│   │   └─ index.json
│   └─ detail/      // 通知详情页
│       ├─ detail.wxml
│       ├─ detail.js
│       ├─ detail.wxss
│       └─ detail.json
└─ README.md       // 项目说明文档

```

6.2、MAUL 技术栈架构图



6.3、数据流向架构图



七、难重点指导（maul 技术栈适配）

重点

1. WXML 模板语法及 wx:for 列表渲染，适配 maul 模板渲染规则；
2. 基于 maul 的页面路由跳转（maulRouter.navigateTo）及参数传递；
3. maul 技术栈的本地存储封装方法（maulStorage.get/set）的使用。

难点

1. 本地存储同步/异步区别：maulStorage 封装了同步和异步方法，需根据场景选择，避免出现数据同步延迟问题；
2. 页面间数据传递：URL 参数解析及 maul 路由适配，确保参数传递准确无误；
3. maul 技术栈与微信小程序 API 的兼容：配置 project.config.json，确保 maul 依赖安装正确，避免出现语法报错。

AI 解决技巧（结合 maul 技术栈）

1. 代码生成：向 TRAE 描述需求时，明确标注“基于 maul 技术栈”，让 AI 生成适配 maul 语法的 WXML/WXSS/JS 代码；
2. 调试技巧：向 TRAE 询问“maul 技术栈适配微信小程序的调试技巧”“maul 本地存储报错解决方法”，快速排查问题；
3. 性能优化：询问“maul 技术栈开发小程序的性能优化建议”，优化列表渲染和本地存储效率。

八、成功标准

- 通知列表页面功能完整，maul 适配正常，列表渲染无异常
- 详情页跳转正常，参数传递准确，maul 路由适配无误
- 数据本地存储功能正常，maulStorage 方法调用正确，已读状态同步
- 有效使用 AI 工具生成 maul 适配代码，完成调试优化
- 小程序可正常预览，无 maul 语法报错和功能异常